

Examen de seconde session SRCS Master 1 SAR

Jonathan Lejeune

Septembre 2020

2 heures – **Tout document papier interdit**
Tout appareil électronique interdit

- Le barème est sur 20 points et est donné à titre indicatif.
- Dans le code java demandé vous ne gérerez pas les imports ni les exceptions sauf si c'est explicitement demandé
- Soyez synthétique et précis dans vos réponses
- Il vous est conseillé de lire attentivement les exercices entièrement avant de composer.
- Vous trouverez à la fin du sujet :
 - ◇ une annexe résumant les champs principaux des principales classes java pour traiter les I/O en mode octet.
 - ◇ une annexe sur l'API de réflexivité de Java

Exercice 1 – Questions de cours (7 points)

Question 1 $\frac{1}{2}$ point

Donnez la définition d'un système réparti.

Question 2 $\frac{1}{2}$ point

Nous disposons d'une variable `ois` de type `ObjectInputStream` correctement instanciée. À l'appel de la méthode `ois.readObject()` l'exception `ClassNotFoundException` est jetée. Que faudrait-il faire pour éviter ceci ?

Question 3 $\frac{1}{2}$ point

Qu'est qu'un protocole sans état ? Donnez un exemple.

Question 4 $\frac{1}{2}$ point

Quel est le patron de conception fondamental utilisé dans le concept des RPC ?

Question 5 $\frac{1}{2}$ point

Qu'est ce que la sérialisation ?

Question 6 $\frac{1}{2}$ point

Sur un serveur multi-threadé quel est l'avantage d'utiliser un pool de thread dynamique par rapport à un pool statique ?

Question 7 1 point

Donner 4 manières d'obtenir une référence sur un objet de type `Class`

Question 8 $\frac{1}{2}$ point

Le code suivant peut-il entraîner un interblocage ? Si non justifier pourquoi, si oui donner un scénario où cela se produit.

```

public class X {
    private Integer i=0;
    private Integer j=0;
    public void f(){
        synchronized(this){
            i++;
            g();
        }
    }

    public void g(){
        synchronized(this){
            j++;
        }
    }
}
} //fin classe

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

Question 9 1/2 point

Dans le code de la question précédente, que se passerait-il si on remplaçait les `synchronized(this)` par `synchronized(i)` ?

Question 10 1/2 point

Quel est l'avantage d'un service web par rapport à un service RMI ?

Question 11 1 point

Qu'est ce qu'une signature numérique ?

Question 12 1/2 point

Les arguments d'un service RMI sont-ils passés par copie ou par référence ? **Justifiez.**

Exercice 2 – Exécution périodique de méthode (5 points)

On souhaite définir un mécanisme permettant d'exécuter périodiquement des méthodes. Ce mécanisme pourrait par exemple servir à logger périodiquement des statistiques ou bien à envoyer périodiquement des messages à une machine (mécanisme de heartbeat).

Pour mettre en place ce mécanisme nous allons utiliser une annotation Java `Periodic` sur des méthodes. La période d'exécution sera renseignée en temps qu'attribut entier de l'annotation et représentera un temps en millisecondes. Pour simplifier, on supposera que les méthodes sur lequel on appliquera cette annotation seront

- des méthodes d'instance
- de visibilité publique
- sans paramètre
- sans valeur de retour

Question 1 1 point

Donnez le code l'annotation `Periodic`

Soit la classe `PeriodicController` suivante :

```

public class PeriodicController {
    private List<Object> objects= new ArrayList<>();
    public PeriodicController(Object... objs) {

```

1
2
3

<code>for(Object o : objs) {</code>	4
<code>objects.add(o);</code>	5
<code>}</code>	6
<code>}</code>	7
<code>public boolean begin(){</code>	8
<code> //????</code>	9
<code>}</code>	10
<code>public void end(){</code>	11
<code> //????</code>	12
<code>}</code>	13
<code>}</code>	14

La méthode `begin` permet de démarrer l'exécution périodique de toute méthode annotée comme tel dans l'ensemble des objets de la liste. Elle retourne vrai si au moins une tâche périodique a été lancée, faux sinon (c'est à dire aucun objet de la liste ne possède de méthode annotée `Periodic`).

La méthode `end` permet d'arrêter l'ensemble des exécutions périodiques lancées par la méthode `begin`. On sortira de la méthode `end` une fois que l'on sera certain que toutes les tâches périodiques soient terminés.

Par exemple si on considère le code suivant :

<code>public class Truc {</code>	1
<code> @Periodic(1000)</code>	2
<code> public void f(){ System.out.println("Truc.f"); }</code>	3
<code> @Periodic(2000)</code>	4
<code> public void g(){ System.out.println("Truc.g"); }</code>	5
<code>}</code>	6
<code>public class Machin {</code>	7
<code> @Periodic(4000)</code>	8
<code> public void f(){ System.out.println("Machin.f"); }</code>	9
<code>}</code>	10
<code>public static void main(String[] args){</code>	11
<code> PeriodicController pc = new PeriodicController(new Machin(), new Truc());</code>	12
<code> if(pc.begin()){</code>	13
<code> //traitement quelconque</code>	14
<code> pc.end();</code>	15
<code> }</code>	16
<code>}</code>	17
	18
	19
	20
	21
	22
	23

peut produire l'affichage suivant

```
Truc.f
Truc.g
Machin.f
Truc.f
Truc.f
Truc.g
Truc.f
```

```
Truc.f
Truc.g
Machin.f
Truc.f
Truc.f
Truc.g
Truc.f
...
```

Question 2 4 points

Donner le code des méthodes `begin` et `end`. Il est demandé de gérer l'interruption `InterruptedException`.

Exercice 3 – Flux d'objets (4 points)

Nous considérons la classe `Transformer` suivante :

<code>public class Transformer {</code>	1
<code> Object transform(Object input){</code>	2
<code> /* du code sans importance */</code>	3
<code> }</code>	4
<code>}</code>	5

Elle offre une méthode qui, pour un objet donné en paramètre, produit un nouvel objet. Nous souhaitons que la fonctionnalité de transformation offerte par la classe `Transformer` soit accessible à distance. Pour ce faire, nous allons déployer une JVM serveur qui se met en attente de connexion de client (on se basera donc sur TCP). Lorsqu'un client se connecte au service (il n'est pas demandé de faire du multi-threading), le serveur se met en attente de réception d'objets. Protocolairement, le premier objet que le client doit envoyer est un entier qui est égal au nombre d'objets que le client enverra et que le serveur doit transformer par la suite. Pour chaque objet reçu, le serveur appelle la méthode `transform` et renvoie au client l'objet ainsi produit par la transformation.

En cas de violation de protocole, notamment si le client n'envoie pas en premier lieu un entier, un objet de type `ProtocolViolation` est renvoyé au nœud fautif et la connexion se termine et le serveur se remet en attente d'autres demandes de connexion.

Question 1 $\frac{1}{2}$ point

Est-ce un protocole à états ou sans état ? **Justifiez**

Question 2 $\frac{1}{2}$ point

Nous utiliserons les flux `ObjectInputStream` et `ObjectOutputStream` de l'API des I/O Java (cf. annexe). Quelle(s) modification(s) apporteriez-vous à la classe `Transformer` pour que le compilateur assure qu'il n'y aura pas d'erreur lors de l'exécution dans la transmission des objets ?

Question 3 3 points

Donnez le code Java de la classe du serveur.

Exercice 4 – Barrière de synchronisation en RMI (4 points)

Nous souhaitons développer un service RMI permettant de gérer des barrières de synchronisation entre plusieurs clients. Pour rappel, une barrière de synchronisation permet d'assurer que N processus aient tous passé un point spécifique de leur programme. Ainsi, dans notre cas, chaque client souhaitant se synchroniser sur une barrière de nom a devra attendre jusqu'à ce que N_a clients se soient synchronisés sur la barrière a . On dira que a est terminée lorsque que N_a clients se sont synchronisés dessus. Le service RMI doit donc offrir la possibilité à un client de :

- créer et d'initialiser une barrière en précisant son nom et sa valeur initiale. Si il existe déjà une barrière du même nom alors l'initialisation doit jeter une exception.
- de se synchroniser sur une barrière en spécifiant son nom. Une exception doit être jetée, si le nom spécifié ne concerne aucune barrière existante, ou alors si la barrière est terminée.

On devra assurer une isolation entre les différentes barrières, c'est à dire que les synchronisations sur une barrière x ne devront pas impacter une autre barrière y et vice-versa.

Question 1 4 points

Écrire les interfaces et classes Java permettant de résoudre ce problème.

Java I/O classes

Abstract Classes	
Class	Main public fields
abstract class <code>InputStream</code> implements <code>Closeable</code>	• abstract int <code>read()</code> throws <code>IOException</code> • int <code>read(byte b[])</code> throws <code>IOException</code> • int <code>read(byte b[], int off, int len)</code> throws <code>IOException</code> • void <code>close()</code> throws <code>IOException</code>
abstract class <code>OutputStream</code> implements <code>Closeable</code> , <code>Flushable</code>	• abstract void <code>write(int b)</code> throws <code>IOException</code> • void <code>write(byte b[])</code> throws <code>IOException</code> • void <code>write(byte b[], int off, int len)</code> throws <code>IOException</code> • void <code>flush()</code> throws <code>IOException</code> • void <code>close()</code> throws <code>IOException</code>

Main InputStream sub-classes	
Class	Main public fields
class <code>FileInputStream</code> extends <code>InputStream</code>	• public <code>FileInputStream(String name)</code> throws <code>FileNotFoundException</code> • public <code>FileInputStream(File file)</code> throws <code>FileNotFoundException</code>
class <code>ByteArrayInputStream</code> extends <code>InputStream</code>	• public <code>ByteArrayInputStream(byte buf[])</code> • public <code>ByteArrayInputStream(byte buf[], int offset, int length)</code>
class <code>BufferedInputStream</code> extends <code>FilterInputStream</code>	• public <code>BufferedInputStream(InputStream in)</code>
class <code>DataInputStream</code> extends <code>FilterInputStream</code> implements <code>DataInput</code>	• public <code>DataInputStream(InputStream in)</code> • <code>String</code> <code>readLine()</code> throws <code>IOException</code> • double <code>readDouble()</code> throws <code>IOException</code> • float <code>readFloat()</code> throws <code>IOException</code> • <code>String</code> <code>readUTF()</code> throws <code>IOException</code> • long <code>readLong()</code> throws <code>IOException</code> • int <code>readInt()</code> throws <code>IOException</code> • char <code>readChar()</code> throws <code>IOException</code> • boolean <code>readBoolean()</code> throws <code>IOException</code> • byte <code>readByte()</code> throws <code>IOException</code>
class <code>ObjectInputStream</code> extends <code>InputStream</code> implements <code>ObjectInput</code> , <code>ObjectStreamConstants</code>	• public <code>ObjectInputStream(InputStream in)</code> throws <code>IOException</code> • methods of <code>DataInputStream</code> • final <code>Object</code> <code>readObject()</code> throws <code>IOException</code>, <code>ClassNotFoundException</code>
class <code>PipedInputStream</code> extends <code>InputStream</code>	• public <code>PipedInputStream(PipedOutputStream src)</code> throws <code>IOException</code> • public <code>PipedInputStream()</code> • void <code>connect(PipedOutputStream src)</code> throws <code>IOException</code>

Main OutputStream sub-classes	
Class	Main public fields
class <code>FileOutputStream</code> extends <code>OutputStream</code>	• public <code>FileOutputStream(String name)</code> throws <code>FileNotFoundException</code> • public <code>FileOutputStream(File file)</code> throws <code>FileNotFoundException</code>
class <code>ByteArrayOutputStream</code> extends <code>OutputStream</code>	• public <code>ByteArrayOutputStream()</code> • public <code>ByteArrayOutputStream(int size)</code> • public <code>byte[]</code> <code>toByteArray()</code>
class <code>BufferedOutputStream</code> extends <code>FilterOutputStream</code>	• public <code>BufferedOutputStream(OutputStream out)</code>

class DataOutputStream extends FilterOutputStream implements DataOutput	• public DataOutputStream(OutputStream out) • void writeDouble(double v) throws IOException • void writeFloat(float v) throws IOException • void writeUTF(String str) throws IOException • void writeLong(long v) throws IOException • void writeInt(int v) throws IOException • void writeChar(int v) throws IOException • void writeBoolean(boolean v) throws IOException • void writeByte(int v) throws IOException • void writeBytes(String s) throws IOException
class ObjectOutputStream extends OutputStream implements ObjectOutput, ObjectOutputStreamConstants	• public ObjectOutputStream(OutputStream out) throws IOException • methods of DataOutputStream • void writeObject(Object obj) throws IOException
class PipedOutputStream extends OutputStream	• public PipedOutputStream(PipedInputStream snk) throws IOException • public PipedOutputStream() • void connect(PipedInputStream snk) throws IOException

Class Class<T>

- [java.lang.Object](#)
- `java.lang.Class<T>`

- Type Parameters:

T - the type of the class modeled by this `Class` object. For example, the type of `String.class` is `Class<String>`. Use `Class<?>` if the class being modeled is unknown.

All Implemented Interfaces:

[Serializable](#), [AnnotatedElement](#), [GenericDeclaration](#), [Type](#)

```
public final class Class<T>
    extends Object
    implements Serializable, GenericDeclaration, Type, AnnotatedElement
```

Instances of the `Class` class represent classes and interfaces in a running Java application. An enum is a kind of class and an annotation is a kind of interface. Every array also belongs to a class that is reflected as a `Class` object that is shared by all arrays with the same element type and number of dimensions. The primitive Java types (`boolean`, `byte`, `char`, `short`, `int`, `long`, `float`, and `double`), and the keyword `void` are also represented as `Class` objects. `Class` has no public constructor. Instead `Class` objects are constructed automatically by the Java Virtual Machine as classes are loaded and by calls to the `defineClass` method in the `ClassLoader`.

The following example uses a `Class` object to print the class name of an object:

```
void printClassName(Object obj) {
    System.out.println("The class of " + obj +
        " is " + obj.getClass().getName());
}
```

Modifier and Type

<U> [Class](#)<? extends [Class](#)<U>> `clazz`

U>

Casts this `Class` object to represent a subclass of the class represented by the specified class object.

[I](#) [Class](#)<? extends [Object](#)> `obj`

Casts an object to the class or interface represented by this `Class` object.

`boolean`

[desiredAssertionStatus\(\)](#)

Returns the assertion status that would be assigned to this class if it were to be initialized at the time this method is invoked.

`static Class<?>`

[forName\(String className\)](#)

Returns the `Class` object associated with the class or interface with the given string name.

`static Class<?>`

[forName\(String name, boolean initialize, \[ClassLoader\]\(#\) loader\)](#)

Returns the `Class` object associated with the class or interface with the given string name, using the given class loader.

[AnnotatedType\[\]](#)

[getAnnotatedInterfaces\(\)](#)

Returns an array of `AnnotatedType` objects that represent the use of types to specify superinterfaces of the entity represented by this `Class` object.

[AnnotatedType](#)

[getAnnotatedSuperClass\(\)](#)

Returns an `AnnotatedType` object that represents the use of a type to specify the superclass of the entity represented by this `Class` object.

<A extends [Annotation](#)> [A](#)

[getAnnotation\(Class<A> annotationClass\)](#)

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

[Annotation\[\]](#)

[getAnnotations\(\)](#)

Returns annotations that are *present* on this element.

<A extends [Annotation](#)> [A\[\]](#)

[getAnnotationsByType\(Class<A> annotationClass\)](#)

Returns annotations that are *associated* with this element.

[String](#)

[getCanonicalName\(\)](#)

Returns the canonical name of the underlying class as defined by the Java Language Specification.

[Class](#)<?>[]

[ClassLoader](#)

[Class](#)<?>

[Constructor](#)<T>

[Constructor](#)<?>[]

<A extends [Annotation](#)> [A](#)

[Annotation\[\]](#)

<A extends [Annotation](#)> [A\[\]](#)

[Class](#)<?>[]

[Constructor](#)<T>

[Constructor](#)<?>[]

[Field](#)

[Field\[\]](#)

[Method](#)

[Method\[\]](#)

[Class](#)<?>

[Class](#)<?>

[Constructor](#)<?>

[Method](#)

[IT\[\]](#)

[Field](#)

[getClasses\(\)](#)

Returns an array containing `Class` objects representing all the public classes and interfaces that are members of the class represented by this `Class` object.

[getClassLoader\(\)](#)

Returns the class loader for the class.

[getComponentType\(\)](#)

Returns the `Class` representing the component type of an array.

[getConstructor\(Class<?>... parameterTypes\)](#)

Returns a `Constructor` object that reflects the specified public constructor of the class represented by this `Class` object.

[getConstructors\(\)](#)

Returns an array containing `Constructor` objects reflecting all the public constructors of the class represented by this `Class` object.

[getDeclaredAnnotation\(Class<A> annotationClass\)](#)

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null.

[getDeclaredAnnotations\(\)](#)

Returns annotations that are *directly present* on this element.

[getDeclaredAnnotationsByType\(Class<A> annotationClass\)](#)

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*.

[getDeclaredClasses\(\)](#)

Returns an array of `Class` objects reflecting all the classes and interfaces declared as members of the class represented by this `Class` object.

[getDeclaredConstructor\(Class<?>... parameterTypes\)](#)

Returns a `Constructor` object that reflects the specified constructor of the class or interface represented by this `Class` object.

[getDeclaredConstructors\(\)](#)

Returns an array of `Constructor` objects reflecting all the constructors declared by the class represented by this `Class` object.

[getDeclaredField\(String name\)](#)

Returns a `Field` object that reflects the specified declared field of the class or interface represented by this `Class` object.

[getDeclaredFields\(\)](#)

Returns an array of `Field` objects reflecting all the fields declared by the class or interface represented by this `Class` object.

[getDeclaredMethod\(String name, Class<?>... parameterTypes\)](#)

Returns a `Method` object that reflects the specified declared method of the class or interface represented by this `Class` object.

[getDeclaredMethods\(\)](#)

Returns an array containing `Method` objects reflecting all the declared methods of the class or interface represented by this `Class` object, including public, protected, default (package) access, and private methods, but excluding inherited methods.

[getDeclaringClass\(\)](#)

If the class or interface represented by this `Class` object is a member of another class, returns the `Class` object representing the class in which it was declared.

[getEnclosingClass\(\)](#)

Returns the immediately enclosing class of the underlying class.

[getEnclosingConstructor\(\)](#)

If this `Class` object represents a local or anonymous class within a constructor, returns a `Constructor` object representing the immediately enclosing constructor of the underlying class.

[getEnclosingMethod\(\)](#)

If this `Class` object represents a local or anonymous class within a method, returns a [Method](#) object representing the immediately enclosing method of the underlying class.

[getEnumConstants\(\)](#)

Returns the elements of this enum class or null if this `Class` object does not represent an enum type.

[getField\(String name\)](#)

Returns a `Field` object that reflects the specified public member field of the class or interface

	represented by this <code>Class</code> object.
<code>Field[]</code>	<code>getFields()</code> Returns an array containing <code>Field</code> objects reflecting all the accessible public fields of the class or interface represented by this <code>Class</code> object.
<code>Type[]</code>	<code>getGenericInterfaces()</code> Returns the <code>Types</code> representing the interfaces directly implemented by the class or interface represented by this object.
<code>Type</code>	<code>getGenericSuperclass()</code> Returns the <code>Type</code> representing the direct superclass of the entity (class, interface, primitive type or void) represented by this <code>Class</code> .
<code>Class<?>[]</code>	<code>getInterfaces()</code> Determines the interfaces implemented by the class or interface represented by this object.
<code>Method</code>	<code>getMethod(String name, Class<?>... parameterTypes)</code> Returns a <code>Method</code> object that reflects the specified public member method of the class or interface represented by this <code>Class</code> object.
<code>Method[]</code>	<code>getMethods()</code> Returns an array containing <code>Method</code> objects reflecting all the public methods of the class or interface represented by this <code>Class</code> object, including those declared by the class or interface and those inherited from superclasses and superinterfaces.
<code>int</code>	<code>getModifiers()</code> Returns the Java language modifiers for this class or interface, encoded in an integer.
<code>String</code>	<code>getName()</code> Returns the name of the entity (class, interface, array class, primitive type, or void) represented by this <code>Class</code> object, as a <code>String</code> .
<code>Package</code>	<code>getPackage()</code> Gets the package for this class.
<code>ProtectionDomain</code>	<code>getProtectionDomain()</code> Returns the <code>ProtectionDomain</code> of this class.
<code>URL</code>	<code>getResource(String name)</code> Finds a resource with a given name.
<code>InputStream</code>	<code>getResourceAsStream(String name)</code> Finds a resource with a given name.
<code>Object[]</code>	<code>getSigners()</code> Gets the signers of this class.
<code>String</code>	<code>getSimpleName()</code> Returns the simple name of the underlying class as given in the source code.
<code>Class<? Super T></code>	<code>getSuperclass()</code> Returns the <code>Class</code> representing the superclass of the entity (class, interface, primitive type or void) represented by this <code>Class</code> .
<code>String</code>	<code>getTypeName()</code> Return an informative string for the name of this type.
<code>TypeVariable<Class<T>>[]</code>	<code>getTypeParameters()</code> Returns an array of <code>TypeVariable</code> objects that represent the type variables declared by the generic declaration represented by this <code>GenericDeclaration</code> object, in declaration order.
<code>boolean</code>	<code>isAnnotation()</code> Returns true if this <code>Class</code> object represents an annotation type.
<code>boolean</code>	<code>isAnnotationPresent(Class<? extends Annotation> annotationClass)</code> Returns true if an annotation for the specified type <i>is present</i> on this element, else false.
<code>boolean</code>	<code>isAnonymousClass()</code> Returns true if and only if the underlying class is an anonymous class.
<code>boolean</code>	<code>isArray()</code> Determines if this <code>Class</code> object represents an array class.
<code>boolean</code>	<code>isAssignableFrom(Class<?> c1s)</code> Determines if the class or interface represented by this <code>Class</code> object is either the same as, or is a superclass or superinterface of, the class or interface represented by the specified <code>Class</code> parameter.
<code>boolean</code>	<code>isEnum()</code> Returns true if and only if this class was declared as an enum in the source code.

<code>boolean</code>	<code>newInstance(Object obj)</code> Determines if the specified <code>Object</code> is assignment-compatible with the object represented by this <code>Class</code> .
<code>boolean</code>	<code>isInterface()</code> Determines if the specified <code>Class</code> object represents an interface type.
<code>boolean</code>	<code>isLocalClass()</code> Returns true if and only if the underlying class is a local class.
<code>boolean</code>	<code>isMemberClass()</code> Returns true if and only if the underlying class is a member class.
<code>boolean</code>	<code>isPrimitive()</code> Determines if the specified <code>Class</code> object represents a primitive type.
<code>boolean</code>	<code>isSynthetic()</code> Returns true if this class is a synthetic class, returns false otherwise.
<code>boolean</code>	<code>newInstance()</code> Creates a new instance of the class represented by this <code>Class</code> object.
<code>String</code>	<code>logNameString()</code> Returns a string describing this <code>Class</code> , including information about modifiers and type parameters.
<code>String</code>	<code>toString()</code> Converts the object to a string.

- **Methods inherited from class java.lang.Object**
[`clone`](#), [`equals`](#), [`finalize`](#), [`getClass`](#), [`hashCode`](#), [`notify`](#), [`notifyAll`](#), [`wait`](#), [`wait`](#), [`wait`](#)

java.lang.reflect

Class Field

- [java.lang.Object](#)
 - [java.lang.reflect.AccessibleObject](#)
 - [java.lang.reflect.Field](#)

- All Implemented Interfaces:
[AnnotatedElement](#), [Member](#)

```
public final class Field
extends AccessibleObject
implements Member
```

A `Field` provides information about, and dynamic access to, a single field of a class or an interface. The reflected field may be a class (static) field or an instance field. A `Field` permits widening conversions to occur during a `get` or `set` access operation, but throws an `IllegalArgumentException` if a narrowing conversion would occur.

Modifier and Type	Method and Description
boolean	equals(Object obj) Compares this <code>Field</code> against the specified object.
Object	get(Object obj) Returns the value of the field represented by this <code>Field</code> , on the specified object.
AnnotatedType	getAnnotatedType() Returns an <code>AnnotatedType</code> object that represents the use of a type to specify the declared type of the field represented by this <code>Field</code> .
<T extends Annotation> T	getAnnotation(Class<T> annotationClass) Returns this element's annotation for the specified type. If such an annotation is <i>present</i> , else null.
<T extends Annotation> T[]	getAnnotationsByType(Class<T> annotationClass) Returns annotations that are <i>associated</i> with this element.
boolean	getBoolean(Object obj) Gets the value of a static or instance <code>boolean</code> field.
byte	getBytes(Object obj) Gets the value of a static or instance <code>byte</code> field.
char	getChar(Object obj) Gets the value of a static or instance field of type <code>char</code> or of another primitive type convertible to type <code>char</code> via a widening conversion.
Annotation[]	getDeclaredAnnotations() Returns annotations that are <i>directly present</i> on this element.
Class<?>	getDeclaringClass() Returns the <code>Class</code> object representing the class or interface that declares the field represented by this <code>Field</code> object.
double	getDouble(Object obj) Gets the value of a static or instance field of type <code>double</code> or of another primitive type convertible to type <code>double</code> via a widening conversion.
float	getFloat(Object obj) Gets the value of a static or instance field of type <code>float</code> or of another primitive type convertible to type <code>float</code> via a widening conversion.
Type	getGenericType() Returns a <code>Type</code> object that represents the declared type for the field represented by this <code>Field</code> object.
int	getInt(Object obj) Gets the value of a static or instance field of type <code>int</code> or of another primitive type convertible to type <code>int</code> via a widening conversion.
long	getLong(Object obj) Gets the value of a static or instance field of type <code>long</code> or of another primitive type convertible to type <code>long</code> via a widening conversion.

int	getModifiers() Returns the Java language modifiers for the field represented by this <code>Field</code> object, as an integer.
String	getName() Returns the name of the field represented by this <code>Field</code> object.
short	getShort(Object obj) Gets the value of a static or instance field of type <code>short</code> or of another primitive type convertible to type <code>short</code> via a widening conversion.
Class<?>	getType() Returns a <code>Class</code> object that identifies the declared type for the field represented by this <code>Field</code> object.
int	hashCode() Returns a hashcode for this <code>Field</code> .
boolean	isEnumConstant() Returns <code>true</code> if this field represents an element of an enumerated type; returns <code>false</code> otherwise.
boolean	isSynthetic() Returns <code>true</code> if this field is a synthetic field; returns <code>false</code> otherwise.
void	set(Object obj, Object value) Sets the field represented by this <code>Field</code> object on the specified object argument to the specified new value.
void	setBoolean(Object obj, boolean z) Sets the value of a field as a <code>boolean</code> on the specified object.
void	setByte(Object obj, byte b) Sets the value of a field as a <code>byte</code> on the specified object.
void	setChar(Object obj, char c) Sets the value of a field as a <code>char</code> on the specified object.
void	setDouble(Object obj, double d) Sets the value of a field as a <code>double</code> on the specified object.
void	setFloat(Object obj, float f) Sets the value of a field as a <code>float</code> on the specified object.
void	setInt(Object obj, int i) Sets the value of a field as an <code>int</code> on the specified object.
void	setLong(Object obj, long l) Sets the value of a field as a <code>long</code> on the specified object.
void	setShort(Object obj, short s) Sets the value of a field as a <code>short</code> on the specified object.
String	toGenericString() Returns a string describing this <code>Field</code> , including its generic type.
String	toString() Returns a string describing this <code>Field</code> .

- Methods inherited from class [java.lang.reflect.AccessibleObject](#)**
[getAnnotations](#), [getDeclaredAnnotation](#), [getDeclaredAnnotationsByType](#), [isAccessible](#), [isAnnotationPresent](#), [setAccessible](#), [setAccessible](#)
- Methods inherited from class [java.lang.Object](#)**
[clone](#), [finalize](#), [getClass](#), [notify](#), [notifyAll](#), [wait](#), [wait](#), [wait](#)

Class Method

- [java.lang.Object](#)
 - [java.lang.reflect.AccessibleObject](#)
 - [java.lang.reflect.Executable](#)
 - [java.lang.reflect.Method](#)

- All Implemented Interfaces:

[AnnotatedElement](#), [GenericDeclaration](#), [Member](#)

Modifier and Type	Method and Description
boolean	<code>equals(Object obj)</code> Compares this Method against the specified object.
<code>AnnotationType</code>	<code>getAnnotatedReturnType()</code> Returns an AnnotationType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable.
<code><T extends Annotation></code>	<code>getAnnotation(Class<T> annotationClass)</code> Returns this element's annotation for the specified type if such an annotation is present, else null.
<code>Annotation[]</code>	<code>getDeclaredAnnotations()</code> Returns annotations that are <i>directly present</i> on this element. <code>getDeclaringClass()</code> Returns the Class object representing the class or interface that declares the executable represented by this object.
<code>Class<?></code>	<code>getDefaultValue()</code> Returns the default value for the annotation member represented by this Method instance.
<code>Object</code>	<code>getExceptionTypes()</code> Returns an array of Class objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object.
<code>Type[]</code>	<code>getGenericExceptionTypes()</code> Returns an array of Type objects that represent the exceptions declared to be thrown by this executable object.
<code>Type[]</code>	<code>getGenericParameterTypes()</code> Returns an array of Type objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
<code>Type</code>	<code>getGenericReturnType()</code> Returns a Type object that represents the formal return type of the method represented by this Method object.
<code>int</code>	<code>getModifiers()</code> Returns the Java language <i>modifiers</i> for the executable represented by this object.
<code>String</code>	<code>getName()</code> Returns the name of the method represented by this Method object, as a String.
<code>Annotation[]</code>	<code>getParameterAnnotations()</code> Returns an array of arrays of Annotations that represent the annotations on the formal parameters, in declaration order, of the Executable represented by this object.
<code>int</code>	<code>getParameterCount()</code> Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.
<code>Class<?>[]</code>	<code>getParameterTypes()</code> Returns an array of Class objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
<code>Class<?></code>	<code>getReturnType()</code> Returns a Class object that represents the formal return type of the method represented by this Method object.

<code>TypeVariable<Method>[]</code>	<code>getParameterTypes()</code> Returns an array of TypeVariable objects that represent the type variables declared by the generic declaration represented by this GenericDeclaration object, in declaration order.
<code>int</code>	<code>hashCode()</code> Returns a hashCode for this Method.
<code>Object</code>	<code>invoke(Object obj, Object... args)</code> Invokes the underlying method represented by this Method object, on the specified object with the specified parameters.
boolean	<code>isBridge()</code> Returns true if this method is a bridge method; returns false otherwise.
boolean	<code>isDefault()</code> Returns true if this method is a default method; returns false otherwise.
boolean	<code>isSynthetic()</code> Returns true if this executable is a synthetic construct; returns false otherwise.
boolean	<code>isVarArgs()</code> Returns true if this executable was declared to take a variable number of arguments; returns false otherwise.
<code>String</code>	<code>toGenericString()</code> Returns a string describing this Method, including type parameters.
<code>String</code>	<code>toString()</code> Returns a string describing this Method.
Methods inherited from class java.lang.reflect.Executable getAnnotatedExceptionTypes, getAnnotatedParameterTypes, getAnnotatedReceiverType, getAnnotationsByType, getParameters Methods inherited from class java.lang.reflect.AccessibleObject getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, setAccessible Methods inherited from class java.lang.Object clone, finalize, getClass, notify, notifyAll, wait, wait, wait Methods inherited from interface java.lang.reflect.AnnotatedElement getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAnnotationPresent	

Class Constructor<T>

- [java.lang.Object](#)
 - [java.lang.reflect.AccessibleObject](#)
 - [java.lang.reflect.Executable](#)
 - [java.lang.reflect.Constructor<T>](#)

• Type Parameters:
T - the class in which the constructor is declared
All implemented Interfaces:

[AnnotatedElement](#), [GenericDeclaration](#), [Member](#)

Modifier and Type	Method and Description
boolean	equals(Object obj) Compares this Constructor against the specified object.
AnnotatedType	getAnnotatedReceiverType() Returns an AnnotatedType object that represents the use of a type to specify the receiver type of the method/constructor represented by this Executable object.
AnnotatedType	getAnnotatedReturnType() Returns an AnnotatedType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable.
<T extends Annotation > T	getAnnotation(Class<T> annotationClass) Returns this element's annotation for the specified type if such an annotation is present, else null.
Annotation[]	getDeclaredAnnotations() Returns annotations that are <i>directly present</i> on this element.
Class<T>	getDeclaringClass() Returns the Class object representing the class or interface that declares the executable represented by this object.
Class<?>[]	getExceptionTypes() Returns an array of Class objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object.
Type[]	getGenericTypeTypes() Returns an array of Type objects that represent the exceptions declared to be thrown by this executable object.
Type[]	getGenericParameterTypes() Returns an array of Type objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
int	getModifiers() Returns the Java language modifiers for the executable represented by this object.
String	getName() Returns the name of this constructor, as a string.
Annotation[]	getParameterAnnotations() Returns an array of arrays of Annotations that represent the annotations on the formal parameters, in declaration order, of the Executable represented by this object.
int	getParameterCount() Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.
Class<?>[]	getParameterTypes() Returns an array of Class objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
TypeVariable<Cons tructor<T>	getTypeParameters() Returns an array of TypeVariable objects that represent the type variables declared by the generic declaration represented by this GenericDeclaration object, in declaration order.
int	hashCode() Returns a hashcode for this Constructor.

boolean

[isSynthetic\(\)](#)

Returns true if this executable is a synthetic construct; returns false otherwise.

boolean

[isVarArgs\(\)](#)

Returns true if this executable was declared to take a variable number of arguments; returns false otherwise.

I

[newInstance\(Object... initargs\)](#)

Uses the constructor represented by this Constructor object to create and initialize a new instance of the constructor's declaring class, with the specified initialization parameters.

[String](#)

[toGenericString\(\)](#)

Returns a string describing this Constructor, including type parameters.

[String](#)

[toString\(\)](#)

Returns a string describing this Constructor.

- [Methods inherited from class java.lang.reflect.Executable](#)
[getAnnotatedExceptionTypes\(\)](#), [getAnnotatedParameterTypes\(\)](#), [getAnnotationsByType\(\)](#), [getParameters\(\)](#)
- [Methods inherited from class java.lang.reflect.AccessibleObject](#)
[getAnnotations\(\)](#), [getDeclaredAnnotation\(\)](#), [getDeclaredAnnotationsByType\(\)](#), [isAccessible\(\)](#), [isAnnotationPresent\(\)](#), [setAccessible\(\)](#), [setAccessible\(\)](#)
- [Methods inherited from class java.lang.Object](#)
[clone\(\)](#), [finalize\(\)](#), [getClass\(\)](#), [notify\(\)](#), [notifyAll\(\)](#), [wait\(\)](#), [wait\(\)](#), [wait\(\)](#)
- [Methods inherited from interface java.lang.reflect.AnnotatedElement](#)
[getAnnotations\(\)](#), [getDeclaredAnnotation\(\)](#), [getDeclaredAnnotationsByType\(\)](#), [isAnnotationPresent\(\)](#)